

Introduction

While alternate messaging applications are widespread, adaptation to the TJHSST community is limited by several factors that they fail to control for. Widespread phone and computer bans complicate the use of these messaging applications, further exacerbating this issue. This problem limits inter-class, inter-club, and social communication between some of the top students in the country. A unified solution is needed to replace the variety of fragmented solutions for our community.

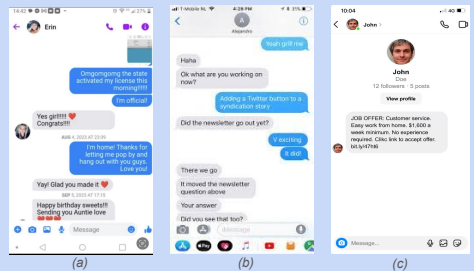


Fig. 1: Messenger, iMessages, and Instagram Messages

Figure 1 depicts (a) Messenger, (b) iMessage, and (c) Instagram Messages, respectively. All three have inherent shortcomings: the search adaptability of Instagram and Messenger is low, and the barrier to contact in iMessage is high. At any rate, all three solutions are lacking in cross-adaptability to the TJHSST environment. JMessage poses a solution to this immediate deficit by integrating with Ion to provide a segmented environment exclusively for, and adapted to, the Thomas Jefferson community.

JMESSAGE: Instant Messaging for the TJHSST Community

Sam Stankiewicz & Nidhish Shakya, Dr. Yilmaz Computer Systems Lab 2026

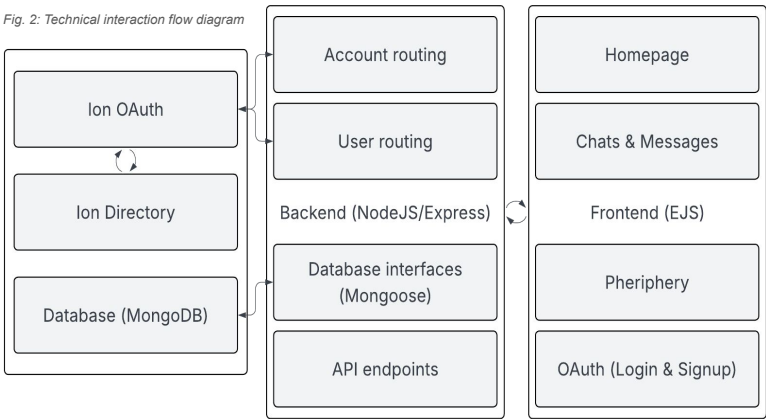
Safety

One potential benefit of our platform is the increased safety implications of operating a messaging application within a closed, containerized environment as opposed an open collection of accounts from any potential outside actor. By restricting our application exclusively to the TJHSST community, we mitigate this effect and develop a novel gain over traditional messaging.

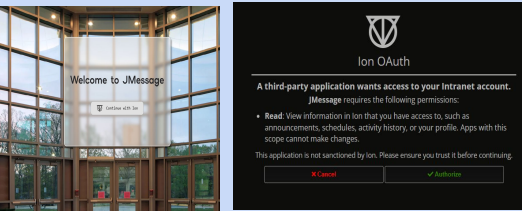
Methods

Our application utilizes a standard NodeJS/ExpressJS application framework with a frontend/backend model. For our database, we utilize MongoDB and interface via Mongoose, a frontend JavaScript framework that we integrate into our backend code. This interaction can be seen in the below Fig. 2, wherein the interfaces between the three components are depicted visually. The database structure was chosen because of the high read/write capacity of MongoDB and low latency architecture. This is preferable to an SQL-based structure, which requires querying an entire database row to retrieve a set of messages. Instead, with the use of MongoDB, we have the ability to individually query specific message tranche, message blocks, from storage.

Fig. 2: Technical interaction flow diagram

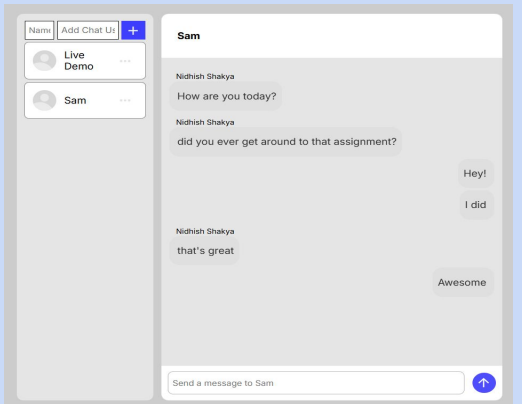


Results



(a): JMessage homepage

(b): JMessage account authentication



(c): JMessage homepage (authenticated state)

Fig. 3: JMessage Application

Our novel solution integrates Ion directory into the traditional messaging interface, as depicted in Fig. 3 (b). With our method, the immediate faults, discussed earlier, posed by traditional messaging applications are remedied by incorporating tools better adapted for the use of TJHSST students.

References

[1] N. Kiran, "Securing Chat applications: Strategies for end-to-end encryption and cloud data protection," World Journal of Advanced Engineering Technology and Sciences, vol. 13, no. 2, pp. 528–540, Dec. 2024, doi: <https://doi.org/10.30574/wjaets.2024.13.2.0634>.

